

Ben Ayers-Glassey

Senior Software Engineer

bayersglassey@gmail.com

(778) 533 2207

Victoria, BC

<https://www.github.com/bayersglassey>

I'm a software engineer with **9 years of professional experience** and a **B.Sc. in mathematics**. I have extensive full-stack experience, creating and maintaining **web applications and RESTful APIs** and implementing **ETL and CI/CD pipelines**. I have experience working with **big data** (Google Cloud) and **deploying containers** using Docker+Kubernetes (via Google's GKE), and have excellent knowledge of **Linux and the network stack**. I'm a Python expert, proficient in Javascript, and have experience with React and Angular on the frontend. I have extensive experience with relational databases, particularly PostgreSQL, including geospatial data (postgis) and vectors (pgvector).

In personal projects, I generally use C and/or Python. I like to study OS internals, create games and small programming languages, and experiment with electronics and embedded software.

I enjoy taking on big problems and solving them a piece at a time. I thrive on projects with a good amount of autonomy, where I can have a big impact on the project's success.

Professional Experience

Senior Back-End Developer

Zesty.AI, US (remote)

Mar 2022 — Jul 2026

Zesty's main product line is climate risk modeling for the property insurance market. I worked on the underlying infrastructure: funneling large amounts of data around, ingesting it from 3rd party sources, caching, implementing business logic, etc.

The ML models involved were mostly vision models, looking at aerial imagery. Many different imagery types and providers were supported, and everything needed to be geolocated. I created and maintained ETL pipelines for many data sources, including imagery and geospatial data.

I'm particularly proud of having rewritten the core of the system, which takes the street address of a property, and returns various kinds of information about it (including e.g. risk scores). Based on the requested information, this system builds a dependency graph, then executes it as fast as possible. Graph nodes represent database queries, API calls, in-house ML models, data transformations, etc. Many graph nodes have "fallback logic" between multiple possible data sources. Users can choose which data sources are allowed, which caches are active, etc. My rewrite replaced a lot of old, ad-hoc Python code, which was very difficult to maintain; whereas the graph theoretical basis of the rewrite made it easy to debug, extend, visualize, test, etc.

Technologies used: Postgres + postgis, GeoJSON, rasterio, GeoTIFF, Python, asyncio, GCS (Google Cloud Storage), BQ (BigQuery), GKE (Google Kubernetes Engine), some Golang

In early 2025, I helped kick off an LLM-based project, ingesting huge amounts of insurance filing data from across the US, and making it easily searchable. I built the initial ETL pipeline (scraping raw data, extracting text, chunking, embedding, loading into database), and the text & vector search tools used by another team's RAG. Some challenges here were around caching: detecting changes to source documents, and only redoing work when needed. We were experimenting with many different tools, so cache keys needed to include the tools, settings, and inputs used; and we needed to be able to run small portions of the ETL pipeline to quickly test new tools.

Technologies used: Python, Postgres + pgvector, Pinecone, embedding models, GCS (Google Cloud Storage), BQ (BigQuery)

Professional Experience (continued)

Full-Stack Developer

Semaphore Solutions, Victoria, BC

Jan 2021 — Mar 2022

Semaphore does contracting work for clients in the health sector, particularly genomics laboratories. I worked with several different teams, some using Python, others Java/Groovy. Almost all projects involved integrating with the same genomic sequencing API (Illumina's Clarity LIMS).

- Early on, I took ownership of an old Python + Flask + Angular "web portal" for doctor's offices. I added various features, including HL7 integration.
- I worked on a high-volume COVID test data processing workflow for one of the largest COVID testing laboratories in the world, Helix (<https://www.helix.com>). The pace was fast and the stakes were high.
- I worked with a team handling small one-off contracts, getting requirements from clients and implementing solutions cleanly & quickly.

Full-Stack Developer

Bambora, Victoria, BC

Jan 2020 — Dec 2020 (1-year contract)

- Worked on a team creating internal tools. Python/Django microservices + React apps.
- Worked on a team maintaining the core system: high-volume financial transaction processing (credit cards, bank transfers, Apple Pay, etc). VB6, COM, ASP, VB.NET, ASP.NET, C#, Java

Full-Stack Developer

Mighty Oaks, Victoria, BC

Oct 2016 — Dec 2019

- Launched e-commerce sites for grocers across Canada, and click-n-collect locker services for Costco & COPA Airlines.
- Backend: Python, Django REST Framework. Frontend: Bootstrap, jQuery

Education

B.Sc. Mathematics

Dalhousie University & Cape Breton University, NS

2013

Personal Projects

Autonomous wildfire investigation drone demo

Closed-source, Apr 2026

Technologies used: Python, Pillow/PIL, MAVLink, C++, QT/QML, UDP, RTP, GStreamer

A friend had a business proposal for training vision models and running them on drones, in order to track wildfires in real-time. I produced a demo showing simulated drones with autonomous fire-investigation behaviour, using real open-source drone software.

The open-source drone software ecosystem is split between two projects: ArduPilot and PX4. They both use the same communication protocol, MAVLink. The software is split between the "autopilot" running on the drone, and the "ground control" running on a tablet or laptop. I created a demo which can be shown entirely from the "ground control" software (QGroundControl). I wrote a drone simulator in Python, which speaks MAVLink, sending telemetry (lat/lng/altitude) and a video stream to ground control, and accepting joystick commands and "missions" (i.e. paths to be patrolled). I modified QGroundControl (C++, QT/QML), adding support for custom MAVLink messages, and adding custom UI elements.

The drone simulator builds a 3d world using real terrain-height data around drone's lat/lng coordinates. "Potential fire" objects are randomly scattered nearby, and when the drone "sees" a potential fire, it switches to an autonomous mode where it flies down to "verify" whether it saw a fire. If so, a custom MAVLink message is sent to ground control, causing a "fire" pin to be dropped on the GPS map.

Basic "swarm" behaviour is supported: multiple drones can be simulated on the same "world", all showing up on the GPS map in ground control, following the same "mission" path, but individually breaking off to investigate separate "potential fires".

I looked at a few Python libraries for rendering 3d scenes, but ended up writing a simple one myself in plain Python, generating frames of video using Pillow/PIL, and streaming them over RTP+UDP using a GStreamer pipeline. Simulation + rendering all ran at 15 FPS, so I was happy!

Personal Projects (continued)

Minimalist multitasking OS for AVR microprocessors

Open-source, 2026

Technologies used: C, AVR assembly, Python

<https://github.com/bayersglassey/minitasks>

In early 2026, I decided to attempt switching from web to embedded/firmware development. I studied the docs and job postings of local companies, and began familiarizing myself with the terminology I found there: SPI, I2C, UART, RS-232/422/485, ModBus, etc. I looked for a cheap hobbyist board I could use to get some experience with these, and bought an Arduino Nano.

Since I have a history of writing tiny programming languages, I figured a good first project would be a tiny OS with tasks written in a toy language. The resulting design was:

- The OS kernel, written in C. Task table & scheduler. Uses the Nano's built-in micro-USB port for loading/starting/stopping/inspecting tasks.
- The toy language, FORTH-like, written in assembly. Primitives for controlling the Nano's built-in LED and getting/sending single characters over UART.
- The client, written in Python. Compiles tasks and interacts with OS kernel over USB.

...and this was successfully implemented. Some challenges and things I learned:

- Debugging serial/USB connection: udev to verify driver stack (CH34x/USB), picocom to debug baud rate, pyserial to understand how to use /dev/ttyUSB0 from Python.
- Learned AVR assembly using official AVR docs, plus decompiling small C functions with avr-objdump.
- Implemented memory dumps via OS kernel and Python client speaking a simple protocol over USB... because the Nano doesn't support JTAG!
- Initially could only run a single task off the UART Rx interrupt. Then got multi-tasking working, using a timer interrupt and implementing context switching in assembly.

Personal Projects (continued)

Game with graphics & physics using discrete algebraic geometry

Open-source, 2018 — present

Technologies used: C, SDL2

<https://github.com/bayersglassey/geom2018>

- Large C codebase, successfully managed over years of development
- Ability to test individual components (graphics, physics, creature animations and behaviour, etc) using text formats and CLI interfaces
- Valgrind used to check for & fix memory leaks
- Emscripten used to compile to WASM & run in the browser

LALANG: interpreted language supporting C extensions

Open-source, 2025

Technologies used: C

<https://github.com/bayersglassey/lalang>

- Stack-based, has functions, classes, iterators... imagine "Python bytecode with a REPL"
- Support for C extensions using dlsym: there is an ABI, against which shared object libraries (.so, .dll, etc) can be compiled, and register their functions/classes/etc when loaded by the VM

FUS: interpreted functional language, able to serve HTTP with uWSGI integration

Open-source, 2018 — 2019

Technologies used: C

<https://github.com/bayersglassey/fus2018>

- All values are immutable, arrays & objects have copy-on-write semantics
- Includes a plugin for uWSGI, allowing functions of the language to process HTTP requests
 - uWSGI is an application server written in C, often used to run Python/WSGI, Perl/PSGI, and ruby/Rack web applications. It can spit out a uwsgi.h file defining an ABI, against which plugins may be compiled as shared object libraries (.so, .dll, etc), and dynamically loaded. The fus2018 plugin turns each HTTP request into a FUS value, passes it to a user-provided FUS function, and turns the resulting FUS value into an HTTP response.